Multimedia Signal processing

Prof. Marco Tagliasacchi

February, 28 2014

1. **Question (8 pts)**:
   Consider two discrete memoryless sources X and Y, whose joint p.d.f. is given by

	b_1	b_2
a_1	0.88	0.02
a_2	0.01	0.09

   (a) Explain how Huffman coding is used to lossless code a discrete source.

   (b) Design three Huffman codes for the source X, obtained by grouping $N = 1, 2$ or 3 symbols, respectively.

   (c) Design a Huffman code that jointly encodes one symbol of X and one symbol of Y.

   (d) Compute the average code length for all the designed codes, and compare the result with both the lower and upper bounds predicted by the theory.

   Solution

   (a) See lecture slides.

   (b) First, let us compute the marginal probability of the symbols of the source X

	b_1	b_2	$p(a)$
a_1	0.88	0.02	0.90
a_2	0.01	0.09	0.10

   - $N = 1$. In this case, a Huffman code is simply given by

symbol	probability	codeword	length
a_1	0.90	0	1
a_2	0.10	1	1

   - $N = 2$. In this case we need to create $2^N = 4$ super-symbols. Since the source X is memoryless, $p(a_i, a_j) = p(a_i)p(a_j)$. Then, a Huffman code is designed by constructing the corresponding tree (not shown in this solution)

symbol	probability	codeword	length
a_1, a_1	0.81	1	1
a_1, a_2	0.09	00	2
a_2, a_1	0.09	010	3
a_2, a_1	0.01	011	3

   - $N = 3$. In this case, we end up with $2^N = 8$ super-symbols

symbol	probability	codeword	length
a_1, a_1, a_1	$0.9^3 = 0.729$	1	1
a_1, a_1, a_2	$0.9^2 \cdot 0.1 = 0.081$	000	3
a_1, a_2, a_1	$0.9^2 \cdot 0.1 = 0.081$	001	3
a_1, a_2, a_2	$0.9 \cdot 0.1^2 = 0.009$	01000	5
a_2, a_1, a_1	$0.9^2 \cdot 0.1 = 0.081$	011	3
a_2, a_1, a_2	$0.9 \cdot 0.1^2 = 0.009$	01001	5
a_2, a_2, a_1	$0.9 \cdot 0.1^2 = 0.009$	01010	5
a_2, a_2, a_2	$0.1^3 = 0.001$	01011	5

(c) When jointly encoding X and Y, a Huffman code is designed starting from super-symbols of two symbols, one per source

symbol	probability	codeword	length
a_1, b_1	0.88	1	1
a_1, b_2	0.02	000	3
a_2, b_1	0.01	001	3
a_2, b_2	0.09	01	2

(d) The average code length are given by

$$L_1 = p(a_1)l(a_1) + p(a_2)l(a_2) = 1 \text{ bit / symbol} \tag{1}$$

$$L_2 = \frac{1}{2}(p(a_1,a_1)l(a_1,a_1)+p(a_1,a_2)l(a_1,a_2)+p(a_2,a_1)l(a_2,a_1)+p(a_2,a_2)l(a_2,a_2)) = 0.645 \text{ bit / symbol} \tag{2}$$

$$L_3 = \frac{1}{3}(p(a_1,a_1,a_1)l(a_1,a_1,a_1) + \ldots p(a_2,a_2,a_2)l(a_2,a_2,a_2)) = 0.5327 \text{ bit / symbol} \tag{3}$$

The entropy of the source X is $H(X) = 0.469$ bit/symbol.
It can be immediately verified that

$$H(X) \leq L_N < H(X) + 1/N \tag{4}$$

When jointly encoding X and Y, the average code length is

$$L_{X,Y} = 1.15 \text{ bits / pair of symbols} \tag{5}$$

The joint entropy of the sources X and Y is equal to

$$H(X,Y) = 0.6543 \text{ bits / pair of symbols} \tag{6}$$

Note that

$$H(X,Y) \leq L_{X,Y} < H(X,Y) + 1 \tag{7}$$

2. **Question (5 pts)**
Illustrate the block diagram of an image codec, describing the role of each functional block.
Solution
See slide set `MMSP_3.1_Image coding`.

3. **Question (5 pts)**
(a) Illustrate forward vs. backward adaptation in lossless coding when using Huffman coding.

(b) Discuss the advantages and disadvantages of both methods, depending on the statistical properties of the underlying source.

Solution

See slide set `MMSP_1.1_Lossless_Coding`.

4. **Questions (5 pts - each answer can be either TRUE or FALSE)**
   (a) Consider a perceptual audio coder.
   - [T] [F] The SMR computed by the perceptual model is always positive.
     FALSE. In the NMT case, a noise masker can mask a tonal masker presented at higher SPL.
   - [T] [F] If a tonal stimulus component is above the absolute threshold of hearing in quite, then it is encoded using $R > 0$ bits
     FALSE. A stimulus might be above the absolute threshold of hearing, but below the aggregated hearing threshold computed by the perceptual model. In this case it is discarder (hence $R = 0$ bits)
   - [T] [F] The perceptual model is a normative part of an audio coding standard.
     FALSE.
   - [T] [F] The MDCT enables to achieve perfect reconstruction and critical sampling.
     TRUE.

   (b) Let I_c and I_r denote, respectively, the current and the reference frame used in motion estimation/-compensation
   - [T] [F] $PSNR(DFD(I_c, I_r)) \geq PSNR(FD(I_c, I_r))$, where DFD and FD denote, respectively, the displaced frame difference and the frame difference.
     TRUE.
   - [T] [F] $PSNR(DFD(I_c, I_r; B_1)) \geq PSNR(DFD(I_c, I_r; B_2))$, if $B_1 > B2$
     FALSE. Decreasing the block size enables to achieve a better estimate of the local motion, hence $PSNR(DFD(I_c, I_r; B_1)) \leq PSNR(DFD(I_c, I_r; B_2))$.
   - [T] [F] When motion estimation is computed at sub-pixel accuracy, the reference frame is spatially interpolated.
     TRUE.

   (c) Consider an image coding system.
   - [T] [F] Zonal quantization allocates the same number of bits to all image blocks.
     TRUE.
   - [T] [F] Threshold quantization uses the same quantization step size for all transform coefficients.
     FALSE. The quantization step size is varied, so as to take into account perceptual distortion.
   - [T] [F] When using threshold quantization it is possible to determine a-priori the number of bits needed to encode an image.
     FALSE.

5. **Question (Matlab - 10pts)**
   You are asked to implement a simplified vocoder, according to the following specifications.
   Split the input signal *offer.wav* into blocks of 20 ms each. Then, for each block
   - Compute $p = 10$ prediction coefficients (hint: use *lpc()*)
   - Estimate the excitation signal.
   - Convert prediction coefficients into reflection coefficients (hint: use *poly2rc()*).
   - Quantize the reflection coefficients using a compander-expander. Use the non-linear function given by the log-area-ratio (LAR)
   $$LAR_i = \log \frac{1 + k_i}{1 - k_i}, \tag{8}$$
   and a quantization step size $\Delta = 1$.

- Compute the speech signal as synthesized by the decoder (for simplicity, assume that the excitation signal is sent to the decoder uncompressed.)

Plot (and play) the original speech and the synthesized speech signal.

Solution

```
%% 2014_02_28_MMSP_exam

clear all
close all
clc

[x, Fs] = wavread('offer.wav');

M = floor(Fs*0.020);

L = length(x);

y = zeros(size(x));

Q = 1;

for i = 1:floor(L/M)

    block = x((i-1)*M + 1: (i-1)*M + M);

    if (sum(block.^2) > 0)

        [a, g] = lpc(block, 10);

        error = filter(a, 1, block);

        k = poly2rc(a);

        LAR = log((1+k)./(1-k));

        LARq = Q*round(LAR./Q);

        kq = (exp(LARq) - 1)./(exp(LARq) + 1);

        aq = rc2poly(kq);

        blockq = filter(1, aq, error);

        y((i-1)*M + 1: (i-1)*M + M) = blockq;
    end
end

plot(x, 'b')
hold on
plot(y, 'r--')
hold off

player = audioplayer(y, Fs); play(player)
```